AI 2002: Artificial Intelligence

Constraint Satisfaction Problems

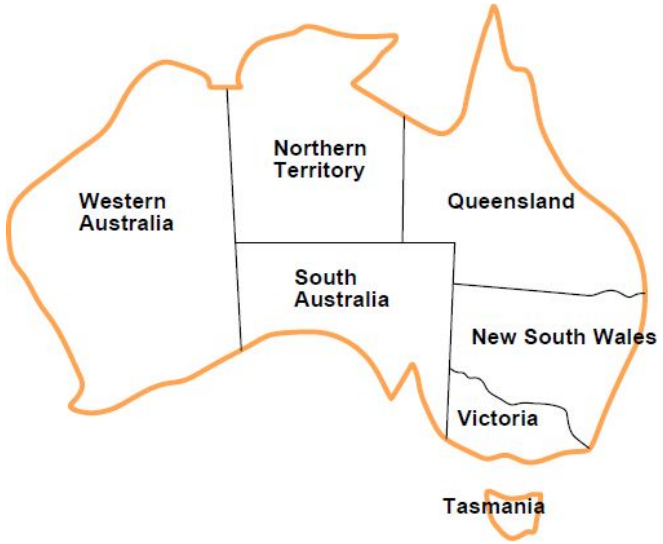
Spring 2026

[These slides (mostly) are based on those created by Dan Klein and Pieter Abbeel (ai.berkeley.edu).]

[Some slides are by Mr. Sandesh Kumar, and Ms. Anaum Hamid, FAST-NUCES, Karahi]

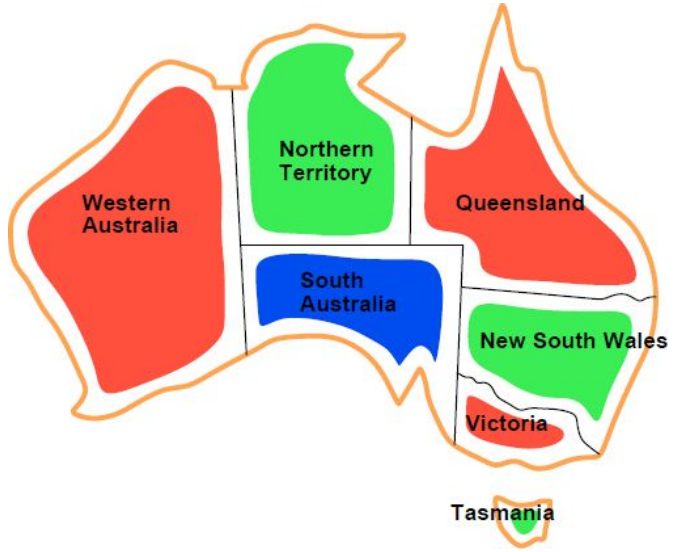
Constraint Satisfaction Problems

- Standard search problems:
  - State is a “black box”: arbitrary data structure
  - Goal test can be any function over states
  - Successor function can also be anything



- Constraint satisfaction problems (CSPs):
  - A special subset of search problems
  - State is defined by variables X_i with values from a domain Di
  - Goal test is a set of constraints specifying allowable combinations of values for subsets of variables

- Allows useful general-purpose algorithms with more power than standard search algorithms



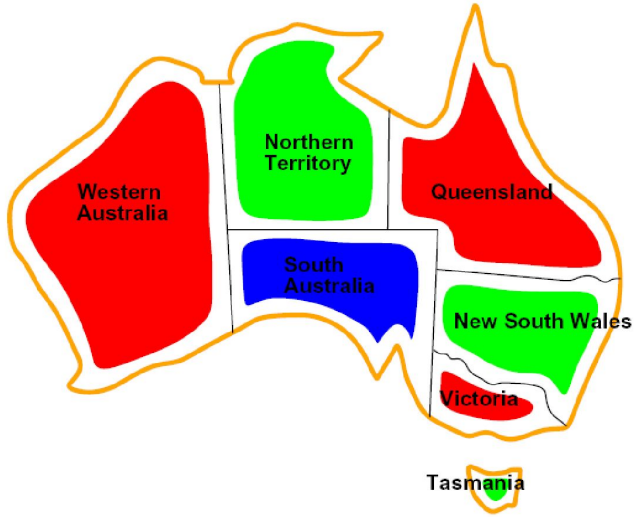
CSP Example: Map Coloring

- Variables: WA, NT, Q, NSW, V, SA, T
- Domains: $D = \{red, green, blue\}$
- Constraints: adjacent regions must have different colors

  Implicit: $WA \neq NT$

  Explicit: $(WA,NT) \in \{(red,green), (red,blue),\ldots\}$
- Solutions are assignments satisfying all constraints, e.g.:

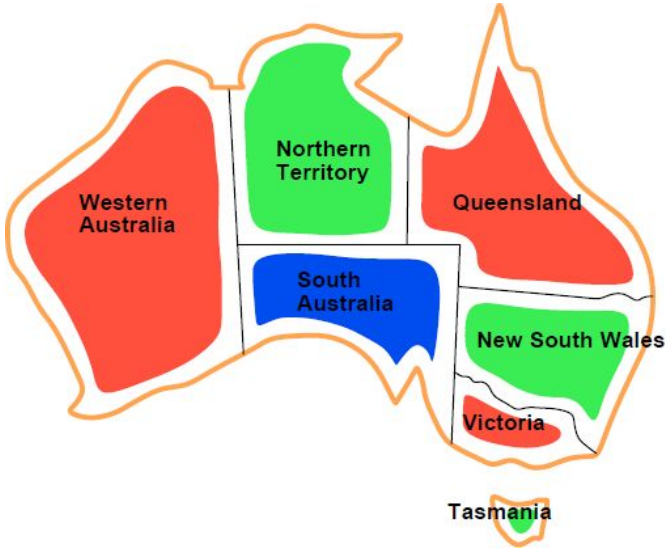
  {WA=red, NT=green, Q=red, NSW=green, V=red, SA=blue, T=green}



- Since there are nine places where regions border, there are nine constraints:
- C = {SA ≠ WA, SA ≠ NT, SA ≠ Q, SA ≠ NSW, SA ≠ V, WA ≠ NT, NT ≠ Q,Q ≠ NSW, NSW ≠ V }

Constraint Satisfaction Problems

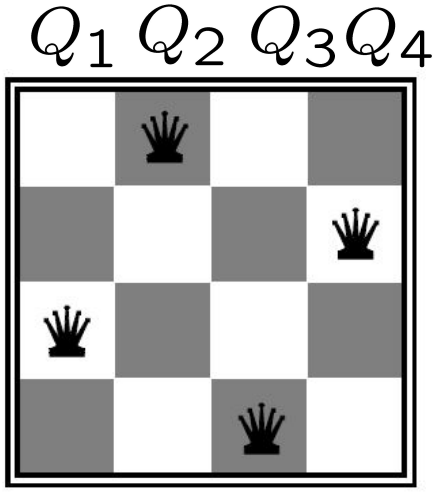
- Each **state in a CSP** is defined **by an assignment of values to some or all of the variables**, {Xi=vi,Xj= vj , . . .}.



- **Consistent (or legal) assignment**
  - An assignment that does not violate any constraints
- **Complete assignment**
  - **A**n assignment in which every variable is assigned.
- A **solution** to a CSP is a consistent, complete assignment.

- A **partial assignment** is one that assigns values to only some of the variables.

CSP Example: N-Queens



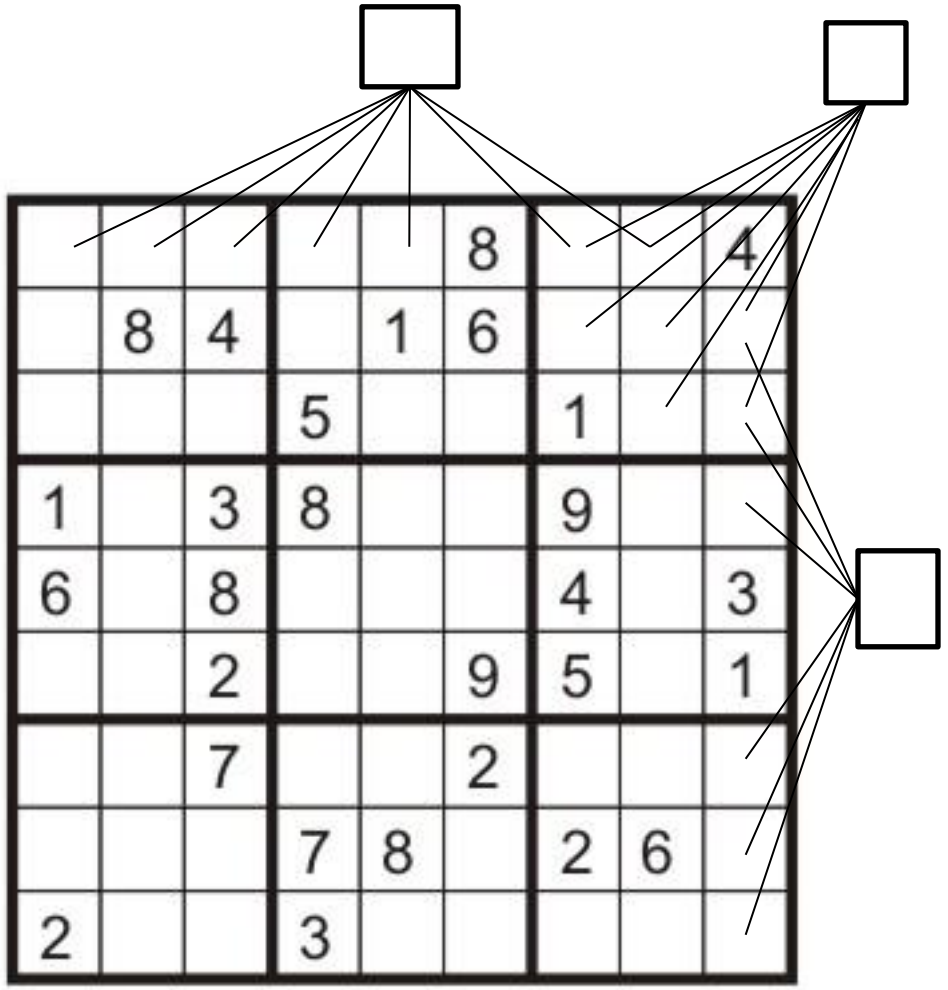
- Variables: Q_k
- Domains: $\{1, 2, 3, \ldots N\}$
- Constraints:

Implicit: $\forall i, j$ non-threatening(Q_i, Q_j)

Explicit: $(Q_1, Q_2) \in \{(1, 3), (1, 4), \ldots\}$

$\ldots$

CSP Example: Sudoku



- Variables:
  - cells (A1-I9)
- Domains:
  - {1,2,…,9}
- Constraints:

  9-way alldiff for each column

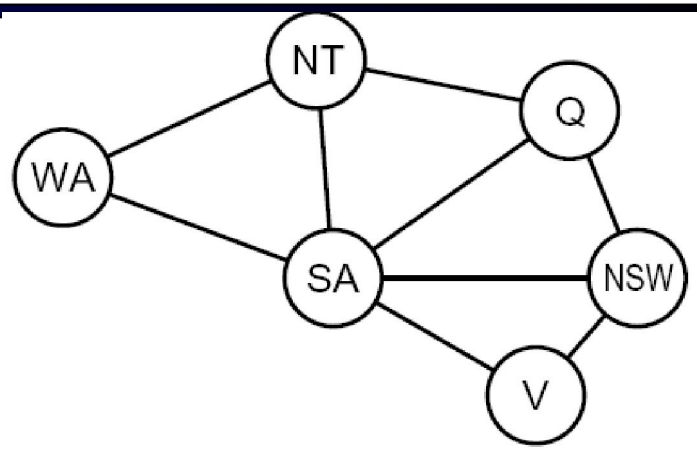
  9-way alldiff for each row

  9-way alldiff for each region

  Some squares already filled in (unary constraints)

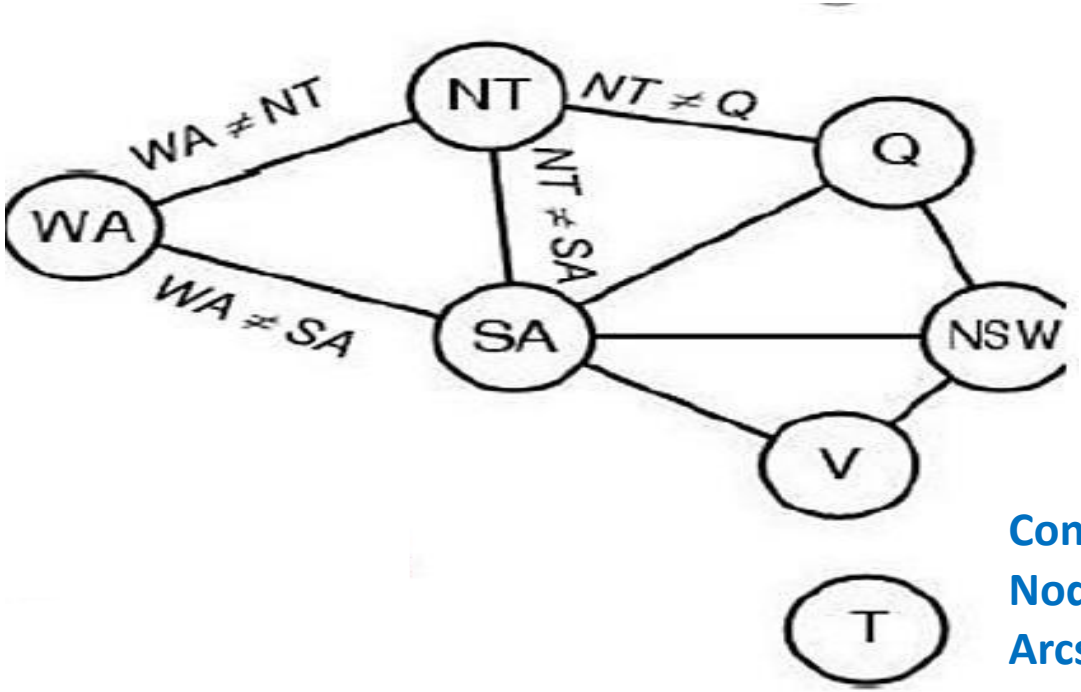
Constraint Graphs

- It can be helpful to visualize a CSP as a constraint graph.
- The **nodes** of the constraint graph correspond to **variables** of the problem,
- A link of the constraint graph connects two variables in a constraint. (**Binary CSP**)
- General-purpose CSP algorithms use the graph structure to speed up search.



	WA	NT	SA	Q	NSW	V
WA		1	1	0	0	0
NT	1		1	1	0	0
SA	1	1		1	1	1
Q	0	1	1		1	0
NSW	0	0	1	1	0	1
V	0	0	1	0	1	0

CSP Representation of Map Coloring



Constraint Graph
Nodes are Variables
Arcs are Constraint

- **Variables:** WA, NT, Q, NSW, V, SA, T
- **Domains:** Di = {red, green, blue}
- **Constraints:** adjacent regions must have different colors
- e.g., WA ≠ NT, or (WA,NT)

Why formulate a problem as a CSP?

- CSP solvers can be faster than state-space searchers
- Because it can quickly eliminate large portions of the search space.

- For example, if {SA=blue}, none of the 'five' neighboring variables can take on the value blue.
  - Without CSP, a search procedure have to consider 3^5 =243 assignments for the five neighboring variables;
  - With CSP, we never have to consider blue as a value, so we have only 2^5 =32 assignments to look at, a reduction of 87%.

- With CSPs, once we find out that a partial assignment is not a solution, we can immediately discard further refinements of the partial assignment.

Variations on the CSP formalism (types of Variable)

- Discrete Variables
  - Finite domains
    - Map-coloring problems, scheduling with time limits and 8-queens problem
  - Infinite domains (integers, strings, etc.)
    - E.g., job scheduling, variables are start/end times for each job
    - A **constraint language** must be used, i.e. implicit: $\mathrm{WA} \neq \mathrm{NT}$, not explicit.
    - **Linear constraints** on integer variables are solvable. ($2x - 3y = 5$)
    - No algorithm exists for solving (all in general) **nonlinear constraints** on integer variables. ($x^2 + y^2 \leq 25$)
- Continuous variables
  - E.g., start/end times for Hubble Telescope observations
  - Linear constraints solvable in polynomial time by LP methods

Polynomial time (Aside)

Polynomial-time: running time is $O(n^k)$, where k is a constant.

- $T(n)$=3
  - yes
- $T(n)$=n
  - Yes
- $T(n)$=$nlg(n)$
  - yes
- $T(n)$=n^3
  - Yes
- $T(n)$=5^n
  - No
- $T(n)$=n!
  - No

- P problems
  - that can be solved by deterministic Turing machine in polynomial-time.
- NP problems
  - that can be solved by non-deterministic Turing machine in polynomial-time.

Why we study NP (Aside)

- One of the most important reasons is:
  - If you see a problem is NPC, you can stop from spending time and energy to develop a fast polynomial-time algorithm to solve it.
  - Just tell your boss it is a NPC problem
  - How to prove a problem is a NPC problem?

- What if a NPC problem needs to be solved?
  - Buy a more expensive machine and wait (could be 100 years)
  - Turn to approximation algorithms (that produce near-optimal solutions)

Approximation algorithms for NPC problems (Aside)

- Define the cost of the optimal solution as C^*
- The cost of the solution produced by an approximation algorithm is C
- The approximation algorithm is then called a $\rho(n)$-approximation algorithm.

$$\boldsymbol{\rho(n)} \geq max(\frac{C}{C^*}, \frac{C^*}{C})$$

- E.g. If MST of a graph G is 20.
- An algorithm can produce some spanning trees, but they are not MSTs, but their weights are always smaller than 25

What if $\boldsymbol{\rho(n)}$=1?

- What is the approximation ratio? 25/20 = 1.25
- This algorithm is called? A 1.25-approximation algorithm

Variations on the CSP formalism (types of Constraints)

- **Unary Constraint** restricts the value of a single variable.

  SA=green
- **Binary Constraint** relates two variables.

  SA≠NSW
- **Higher-order constraints** (global constraints) involve 3 or more variables.

  cryptarithmetic column constraints
- A CSP can be transformed into a CSP with only binary constraints
- **Preferences (soft constraints)**

  E.g., red is better than green
  - Often representable by a cost for each variable assignment
  - Gives constrained optimization problems

Real-world CSPs

- Assignment problems
  - e.g., who teaches what class
- Timetabling problems
  - e.g., which class is offered when and where?
- Hardware configuration
- Transportation scheduling
- Factory scheduling

- Many real-world problems involve real-valued variables

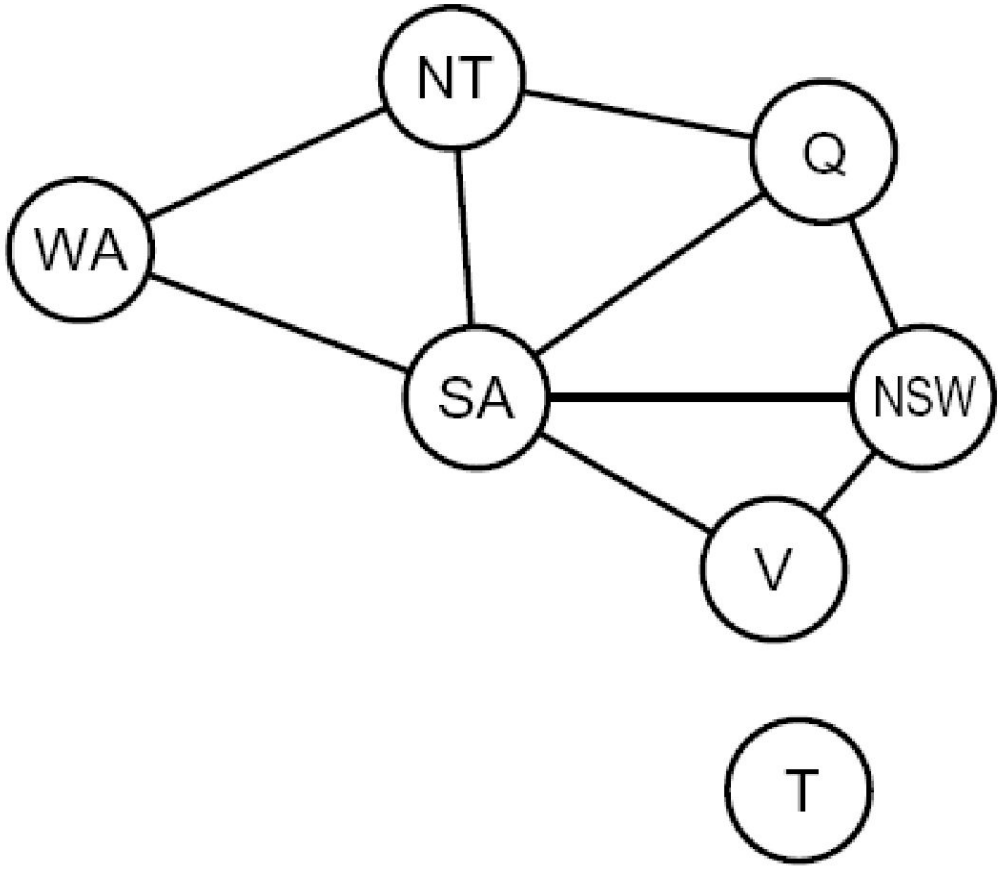
Search Formulation for CSPs

- States
  - defined by the values assigned so far (partial assignments)
- Initial state:
  - the empty assignment, {}
- Successor function:
  - assign a value to an unassigned variable
- Goal test:
  - the current assignment is complete and satisfies all constraints

- We'll start with the straightforward, naïve approach, then improve it

Search Methods

- What would BFS do?
  - Root: Empty assignment: {}
  - Sweep over partial assignments to 1 variable: {WA = R}, {WA = B}, {WA = G}, {NT = R}…
  - Then partial assignments to 2 variables: {WA = R, NT = R}, {WA = R, NT = B}…
  - ...has to go through whole tree!
  - All solutions are at the bottom = bad!



- What would DFS do?
  - Perhaps more sensible? At least it starts by checkin complete solutions

Intelligent Backtracking: Looking Backward

- Consider a fixed variable ordering Q, NSW, V , T, SA, WA, NT. Suppose we have generated the partial assignment {Q=red, NSW =green, V =blue, T =red}.
  When we try the next variable,
- SA, we see that every value violates a constraint(chronological backtracking)
- Recoloring Tasmania cannot possibly resolve the problem with South Australia.
- A more intelligent approach to backtracking is to backtrack to a variable that might fix the problem
- The set (in this case {Q=red ,NSW =green, V =blue, }), is called the **conflict set** for SA.
- In this case, back jumping would jump over Tasmania and try a new value for V .

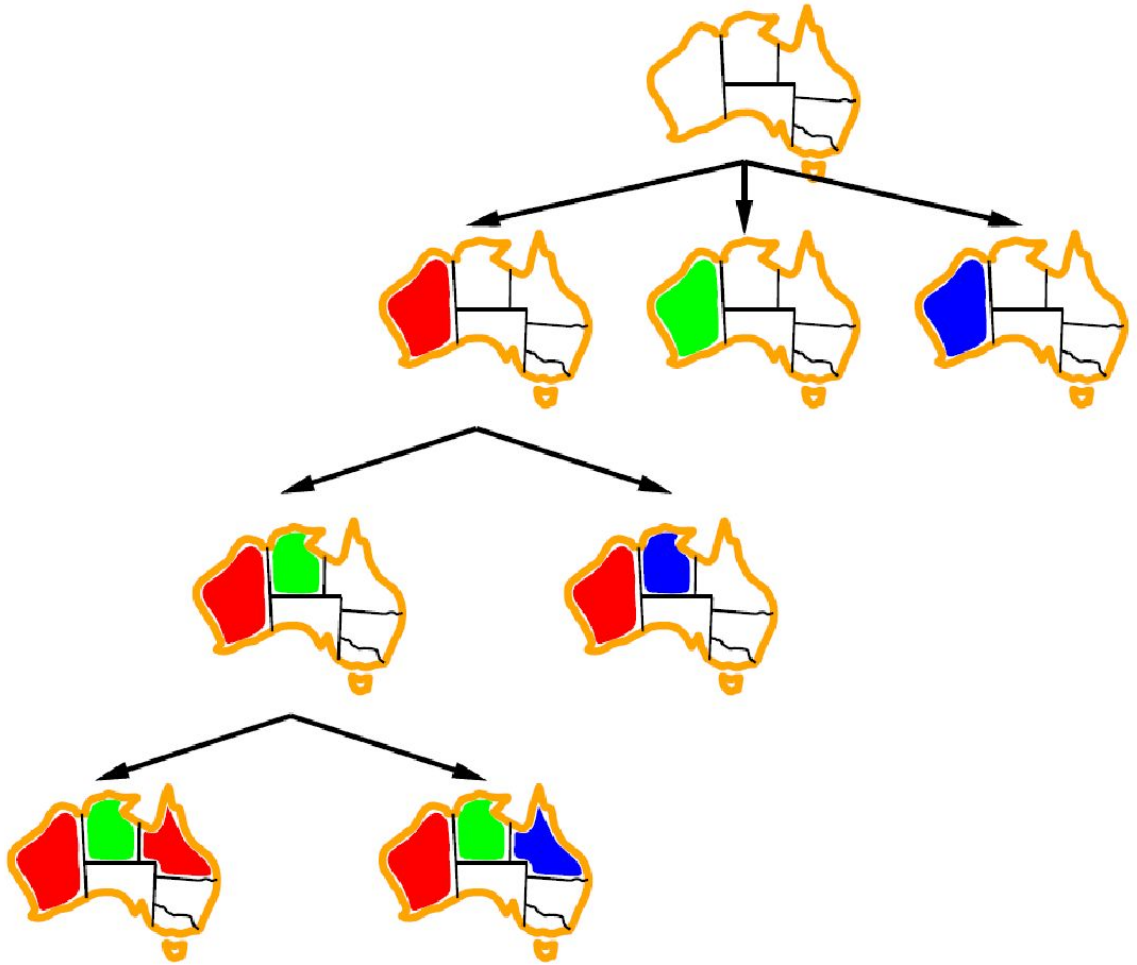
Backtracking search

- Depth-first search for CSPs with single-variable assignments is called backtracking search
- Variable assignments are commutative: [WA = red then NT = green] same as [NT = green then WA = red]
- Only need to consider assignments to a single variable at each node
- Backtracks when variable has no legal value

Backtracking Search

- Backtracking search is the basic uninformed algorithm for solving CSPs
- Idea 1: One variable at a time
  - Variable assignments are commutative
  - i.e., [WA = red then NT = green] same as [NT = green then WA = red]
  - Only need to consider assignments to a single variable at each step
  - With this restriction (commutative property of CSPs), the number of leaves is d^n.
- Idea 2: Check constraints as you go
  - I.e. consider only values which do not conflict with previous assignments
  - Might have to do some computation to check the constraints
  - "Incremental goal test"
- Depth-first search with these two improvements is called *backtracking search* (not the best name)
- Can solve n-queens for $n \approx 25$

Backtracking Example



Backtracking Search

```
function BACKTRACKING-SEARCH(csp) returns solution/failure
    return RECURSIVE-BACKTRACKING({ }, csp)

function RECURSIVE-BACKTRACKING(assignment, csp) returns soln/failure
    if assignment is complete then return assignment
    var ← SELECT-UNASSIGNED-VARIABLE(VARIABLES[csp], assignment, csp)
    for each value in ORDER-DOMAIN-VALUES(var, assignment, csp) do
        if value is consistent with assignment given CONSTRAINTS[csp] then
            add {var = value} to assignment
            result ← RECURSIVE-BACKTRACKING(assignment, csp)
            if result ≠ failure then return result
            remove {var = value} from assignment
    return failure
```

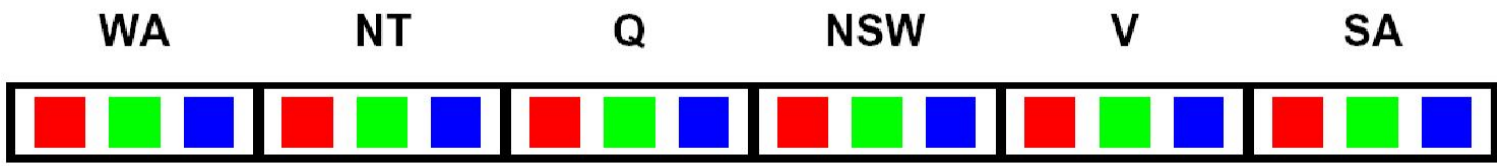
- Backtracking = DFS + variable-ordering + fail-on-violation
- What are the choice points? Order of variables, order of values.

Improving Backtracking

- General-purpose ideas give huge gains in speed
  - (Unlike A* heuristics, these apply *across* many problems)
- Ordering:
  - Which variable should be assigned next?
  - In what order should its values be tried?
- Filtering: Can we detect inevitable failure early?
- Structure: Can we exploit the problem structure?

Filtering: Forward Checking

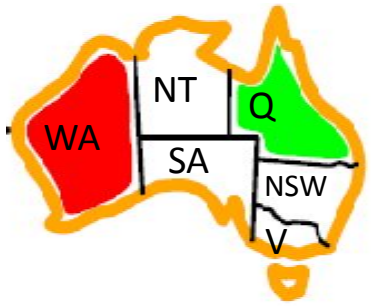
- Filtering: Keep track of domains for unassigned variables and cross off bad options
- Forward checking: Cross off values that violate a constraint when added to the existing assignment

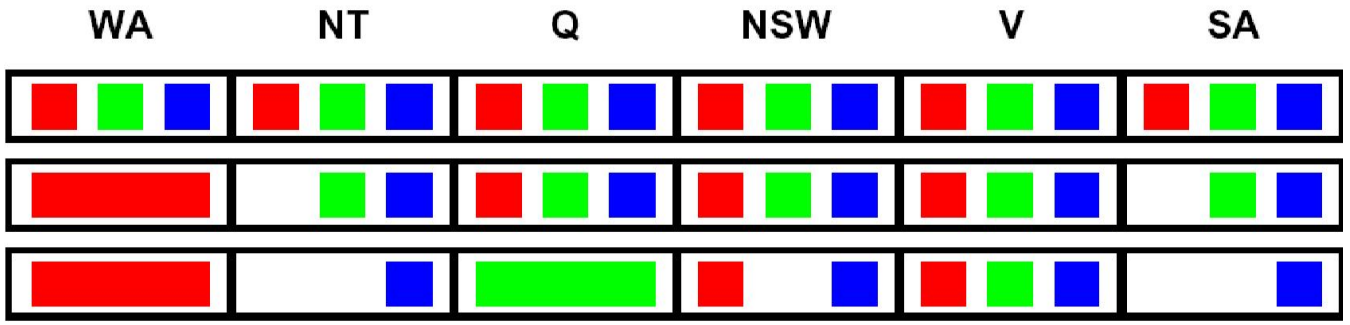


- Forward checking is actually the inference algorithm
- Inference algorithms can infer reductions in the domain of variables before we begin the search

Filtering: Constraint Propagation

- Forward checking propagates information from assigned to unassigned variables, but doesn't provide early detection for all failures:

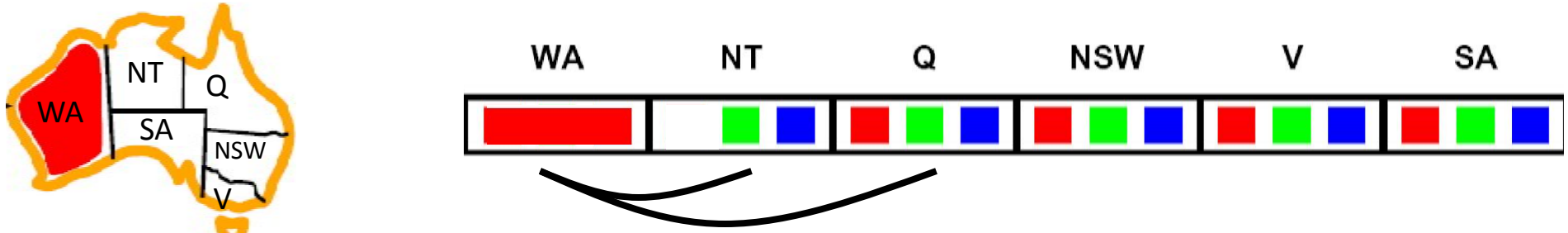




- NT and SA cannot both be blue!
- Why didn’t we detect this yet?
- *Constraint propagation:* reason from constraint to constraint

Consistency of A Single Arc

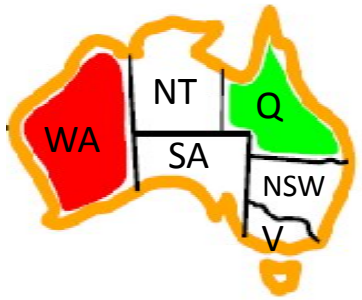
- An arc X → Y is consistent iff for *every* x in the tail there is *some* y in the head which could be assigned without violating a constraint (head = the arrow!)

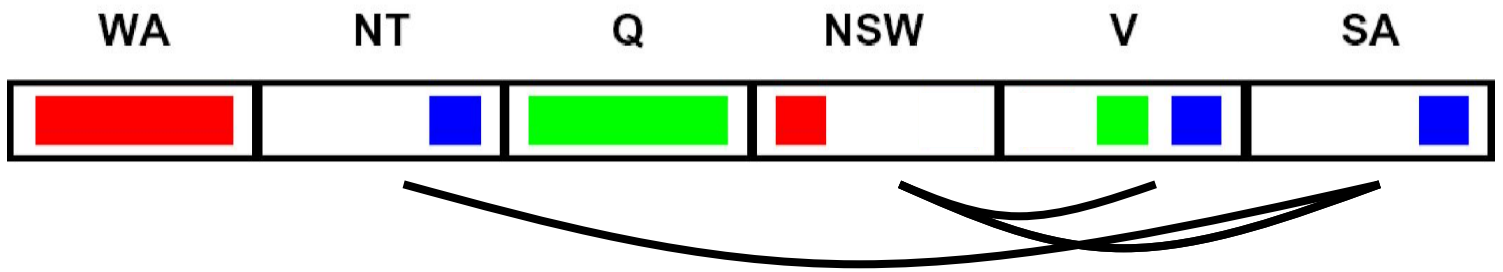


- Forward checking: Enforcing consistency of arcs pointing to each *new assignment*. But other types of arc consistency algorithms are possible!

Arc Consistency of an Entire CSP

- A simple form of propagation makes sure all arcs are consistent:





- Important: If X loses a value, neighbors of X need to be rechecked!
- Arc consistency detects failure earlier than forward checking
- Can be run as a preprocessor or after each assignment
- A graph is arc consistent if all arcs in the graph are consistent
- What's the downside of enforcing arc consistency?

Remember: Delete from the tail!

Enforcing Arc Consistency in a CSP

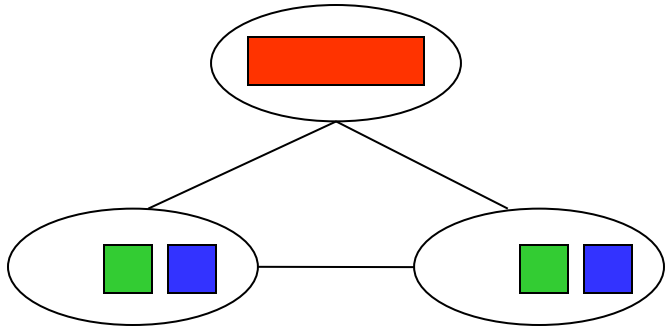
```
function AC-3( csp) returns the CSP, possibly with reduced domains
  inputs: csp, a binary CSP with variables {X_1, X_2, ..., X_n}
  local variables: queue, a queue of arcs, initially all the arcs in csp

  while queue is not empty do
    (X_i, X_j) ← REMOVE-FIRST(queue)
    if REMOVE-INCONSISTENT-VALUES(X_i, X_j) then
      for each X_k in NEIGHBORS[X_i] do
        add (X_k, X_i) to queue

---------------------------------------------------------------------

function REMOVE-INCONSISTENT-VALUES( X_i, X_j) returns true iff succeeds
  removed ← false
  for each x in DOMAIN[X_i] do
    if no value y in DOMAIN[X_j] allows (x,y) to satisfy the constraint X_i ↔ X_j
      then delete x from DOMAIN[X_i]; removed ← true
  return removed
```

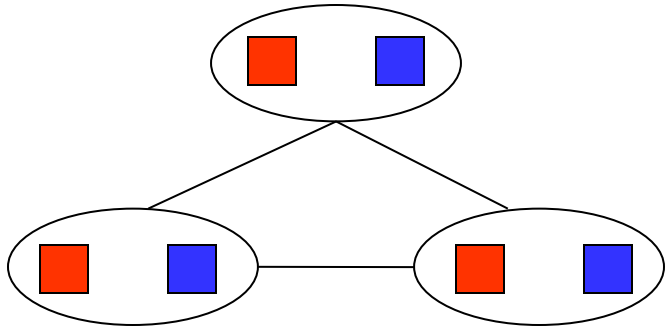
- Runtime: $O(n^2d^3)$, can be reduced to $O(n^2d^2)$
- … but detecting all possible future problems is NP-hard – why? Because it would enable us to solve NP-hard problems (like satisfiability).

Limitations of Arc Consistency

- After enforcing arc consistency:
  - Can have one solution left
  - Can have multiple solutions left
  - Can have no solutions left (and not know it)

- Arc consistency still runs inside a backtracking search!



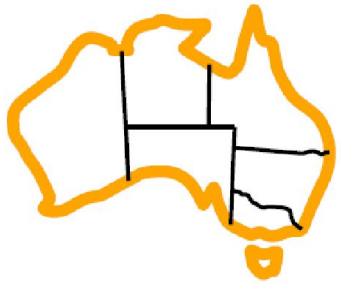


What went wrong here?

Ordering: Minimum Remaining Values

a heuristic for variable selection

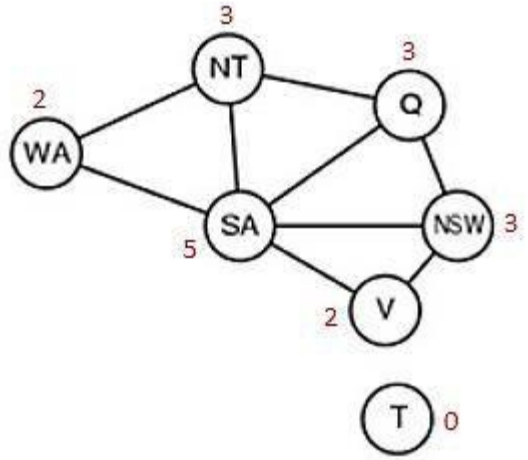
- Variable Ordering: Minimum remaining values (MRV):
  - Choose the variable with the fewest legal left values in its domain



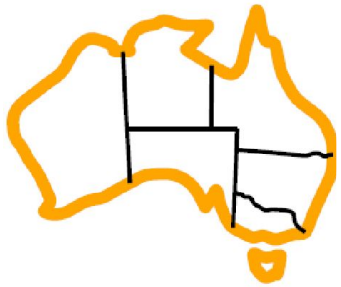
- Why min rather than max?
- "Fail-fast" ordering

Ordering: Degree Heuristic

a heuristic for variable selection



- Tie-breaker among MRV variables
- Degree heuristic:
  - Choose the variable participating in the most constraints on remaining variables

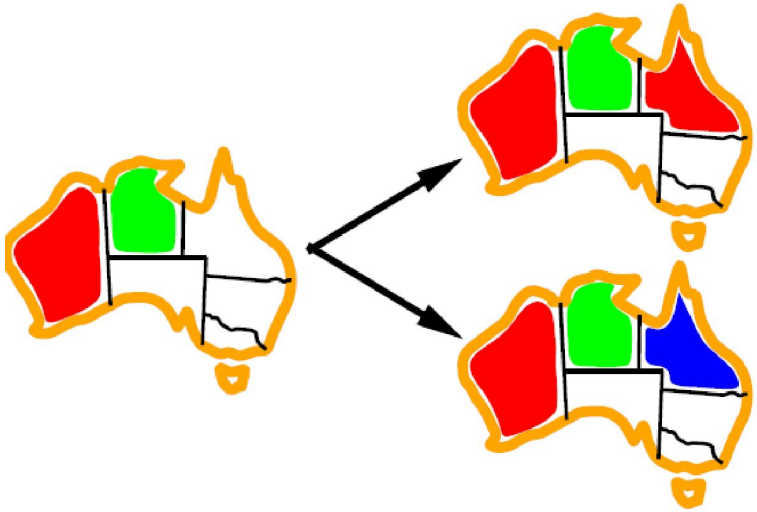


- Why most rather than fewest constraints?

Ordering: Least Constraining Value

a heuristic for value selection

- Value Ordering: Least Constraining Value
  - Given a choice of variable, choose the *least constraining value*
  - I.e., the one that rules out the fewest values in the remaining variables
  - Note that it may take some computation to determine this! (E.g., rerunning filtering)



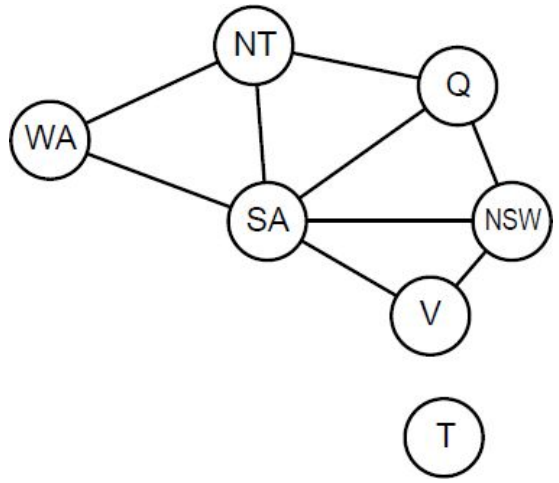
- Why least rather than most?

- Combining these ordering ideas makes 1000 queens feasible

Problem Structure

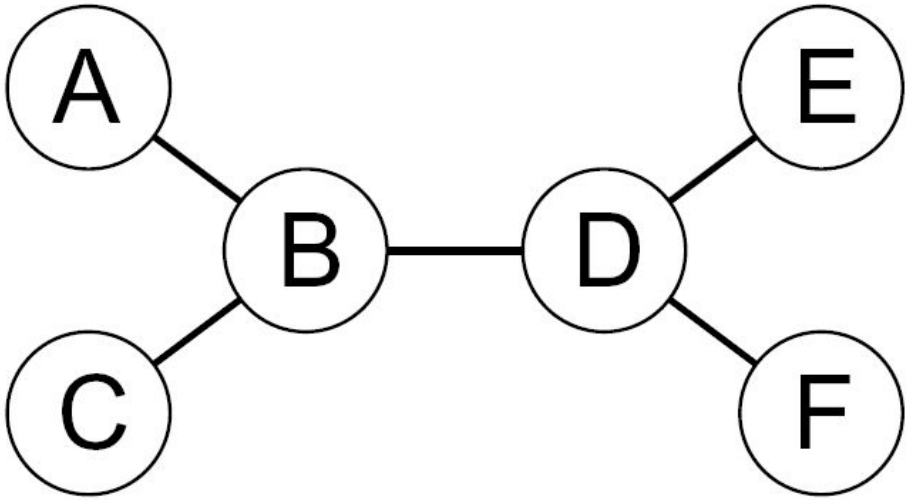
- The structure of the problem can be used to find solutions quickly.
- Coloring Tasmania and coloring the mainland are **independent subproblems**

- Why independent subproblems are important?
  - Suppose each CSP_i has c variables from the total of n variables, where c is a constant.
  - Then there are n/c subproblems, each of which takes at most d^c work to solve, where d is the size of the domain.
  - Hence, the total work is $O(d^c\ n/c)$, which is linear in n.
  - Without decomposition, total work is $O(d^n)$, i.e. exponential in n.



  - E.g., n=80, d=2, c=20
  - 2^{80} = 4 billion years at 10 million nodes/sec
  - (4) * (2^{20}) = 0.4 seconds at 10 million nodes/sec

Tree-Structured CSPs

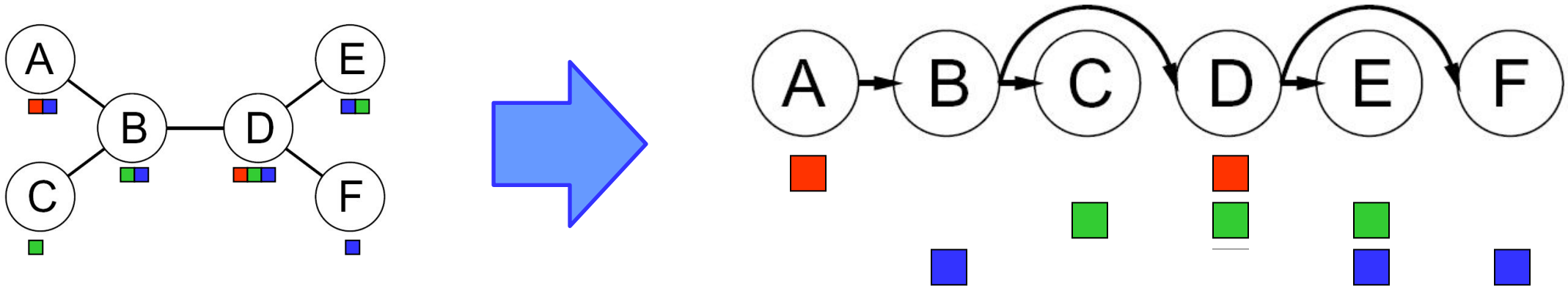


- Theorem: if the constraint graph has no loops, the CSP can be solved in $O(n\ d^2)$ time
  - Compare to general CSPs, where worst-case time is $O(d^n)$

- This property also applies to probabilistic reasoning (later): an example of the relation between syntactic restrictions and the complexity of reasoning

Tree-Structured CSPs

- Algorithm for tree-structured CSPs:
  - Order: Choose a root variable, order variables so that parents precede children



  - Remove backward: For i = n : 2, apply RemoveInconsistent(Parent(X_i),X_i)
  - Assign forward: For i = 1 : n, assign X_i consistently with Parent(X_i)
- Runtime: $O(n\ d^2)$ (why?)

Local Search for CSPs

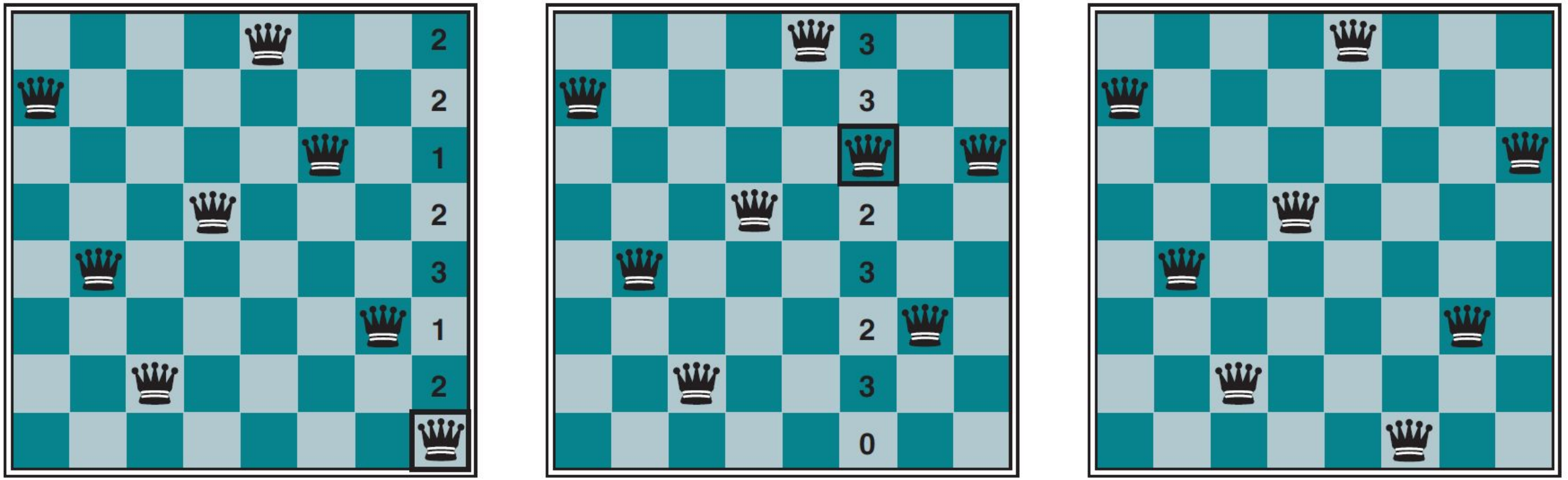


Figure 5.8 A two-step solution using min-conflicts for an 8-queens problem. At each stage, a queen is chosen for reassignment in its column. The number of conflicts (in this case, the number of attacking queens) is shown in each square. The algorithm moves the queen to the min-conflicts square, breaking ties randomly.

Local Search for CSPs

- CSP can be used in an online when the problem changes.
- Consider a scheduling problem for an airline's weekly flights. The schedule may involve thousands of flights and tens of thousands of personnel assignments, but bad weather at one airport can render the schedule infeasible. We would like to repair the schedule with a minimum number of changes.
- This can be easily done with a local search algorithm starting from the current schedule.
- A backtracking search with the new set of constraints usually requires much more time and might find a solution with many changes from the current schedule.
- To conclude: 8-Queens is a CSP problem, but min-conflicts solves it using a local-search (hill-climbing-like) repair strategy rather than systematic backtracking search.

CRYPTARITHMETIC

CSP Example: Cryptarithmetic

- Variables:

  $F\ T\ U\ W\ R\ O\ X_1\ X_2\ X_3$

- Domains:

  $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

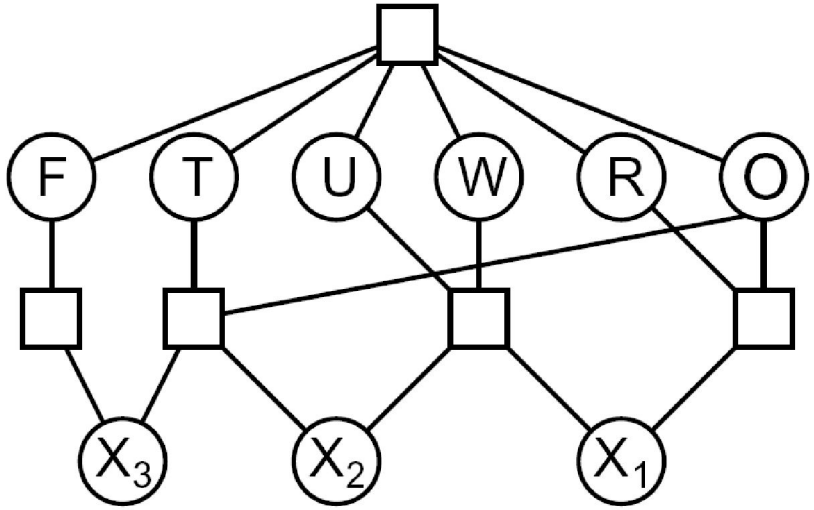
- Constraints:

  alldiff(F, T, U, W, R, O)

  $O + O = R + 10 \cdot X_1$

  $\ldots$

```
  T W O
+ T W O
-------
F O U R
```



Example: Cryptarithmetic

- In a Cryptarithmetic puzzle, each letter stands for a distinct digit; the aim is to find a substitution of digits for letters such that the resulting sum is arithmetically correct.

$$\begin{array}{r} T\ W\ O \\ +\ T\ W\ O \\ \hline F\ O\ U\ R \end{array}$$

- The global constraint **Alldiff** (F, T,U,W,R,O) represents distinction of each letter.
- The constraints on the four columns of the puzzle can be written as n-ary constraints.

$$\begin{aligned} O + O &= R + 10 \cdot C_{10} \\ C_{10} + W + W &= U + 10 \cdot C_{100} \\ C_{100} + T + T &= O + 10 \cdot C_{1000} \\ C_{1000} &= F\ , \end{aligned}$$

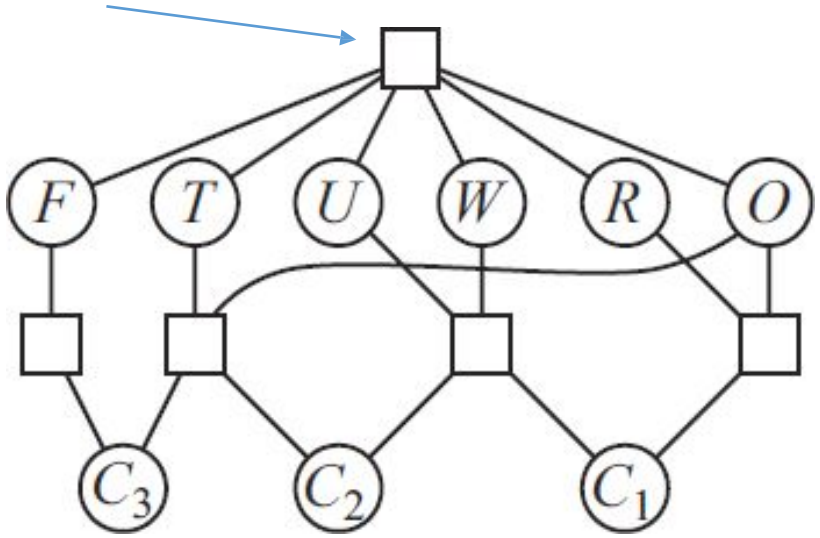
- where C_{10}, C_{100}, and C_{1000} are auxiliary variables representing the digit carried over into the tens, hundreds, or thousands column.

Example: Cryptarithmetic

- Constraints can be represented in a constraint hypergraph
- A hypergraph consists of ordinary nodes and hypernodes which represent n-ary constraints.

$$\begin{array}{r} T\ W\ O \\ +\ T\ W\ O \\ \hline F\ O\ U\ R \end{array}$$

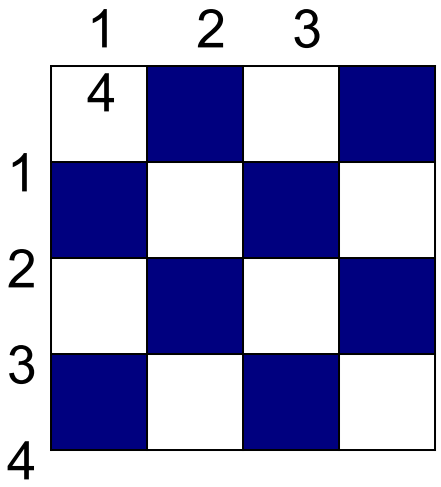
Alldiff (F, T,U,W,R,O)

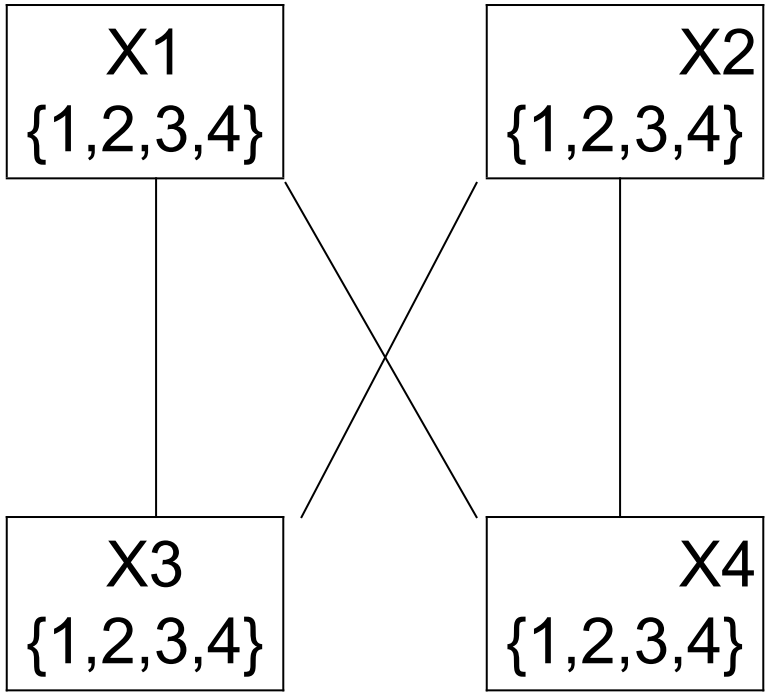


$$O + O = R + 10 \cdot C_{10}$$
$$C_{10} + W + W = U + 10 \cdot C_{100}$$
$$C_{100} + T + T = O + 10 \cdot C_{1000}$$
$$C_{1000} = F\ ,$$

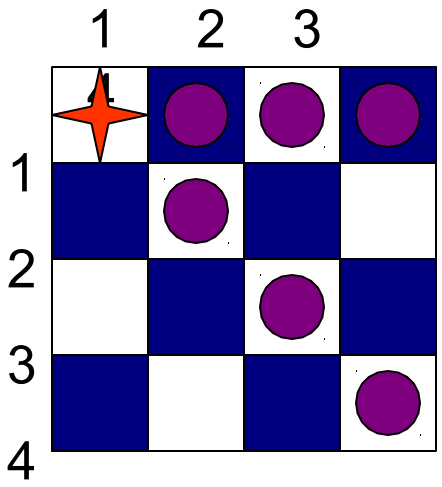
N-QUEEN PROBLEM

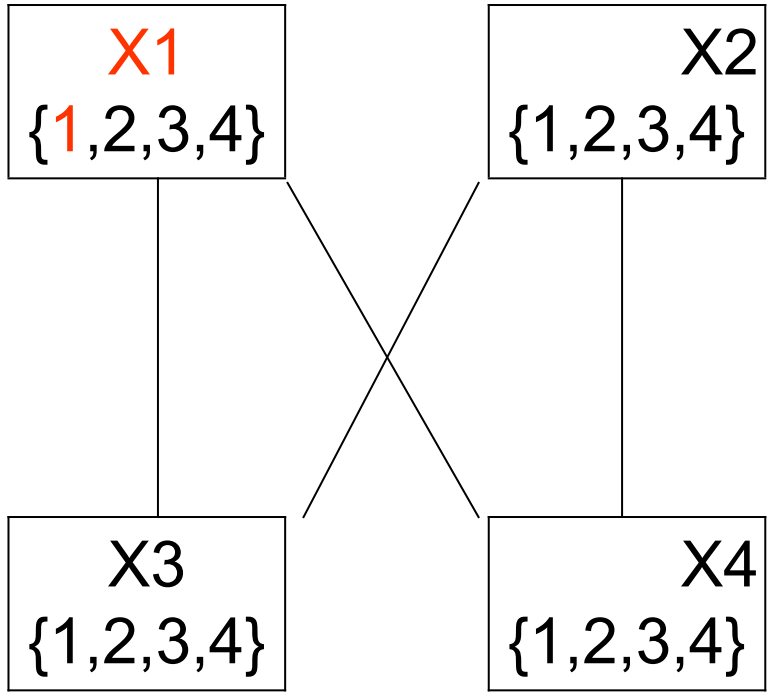
Example: 4-Queens Problem



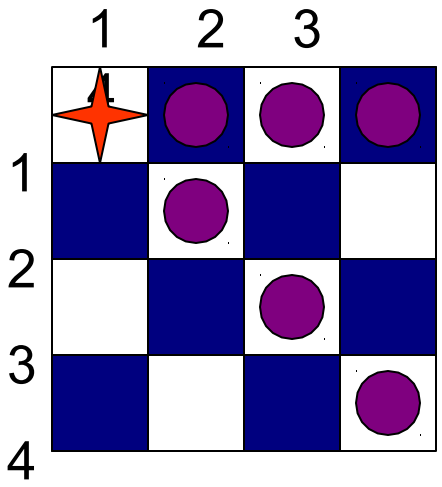


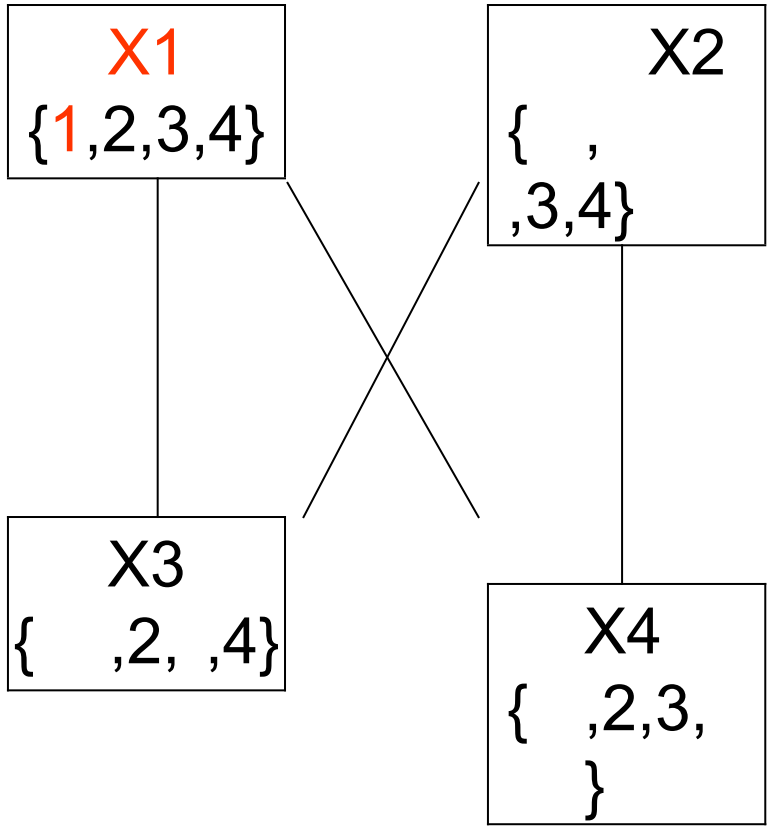
Example: 4-Queens Problem



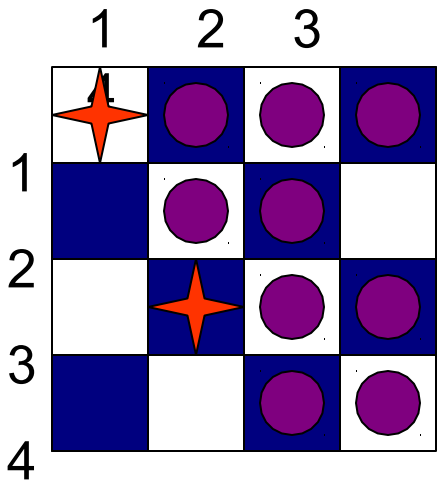


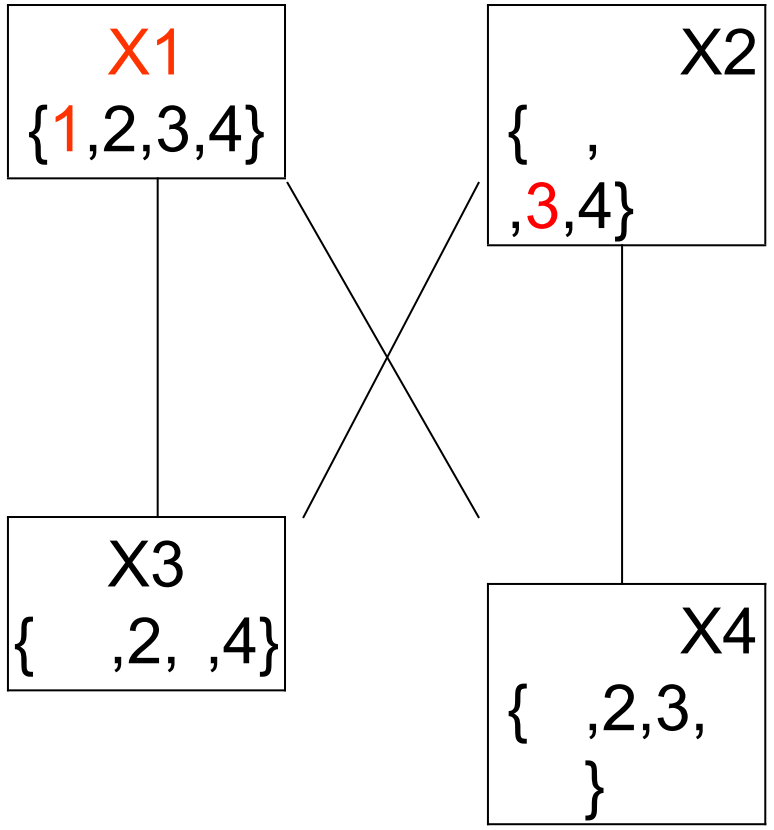
Example: 4-Queens Problem



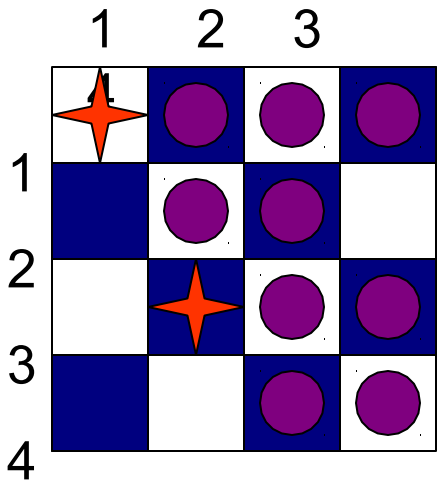


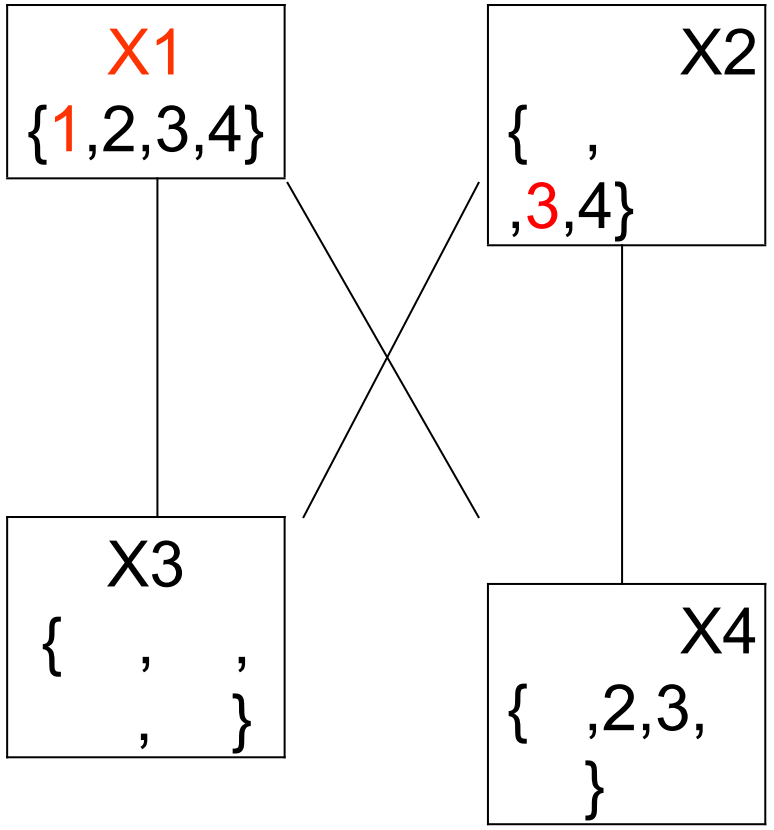
Example: 4-Queens Problem



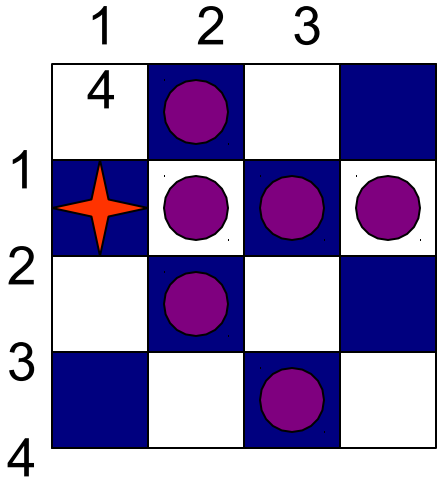


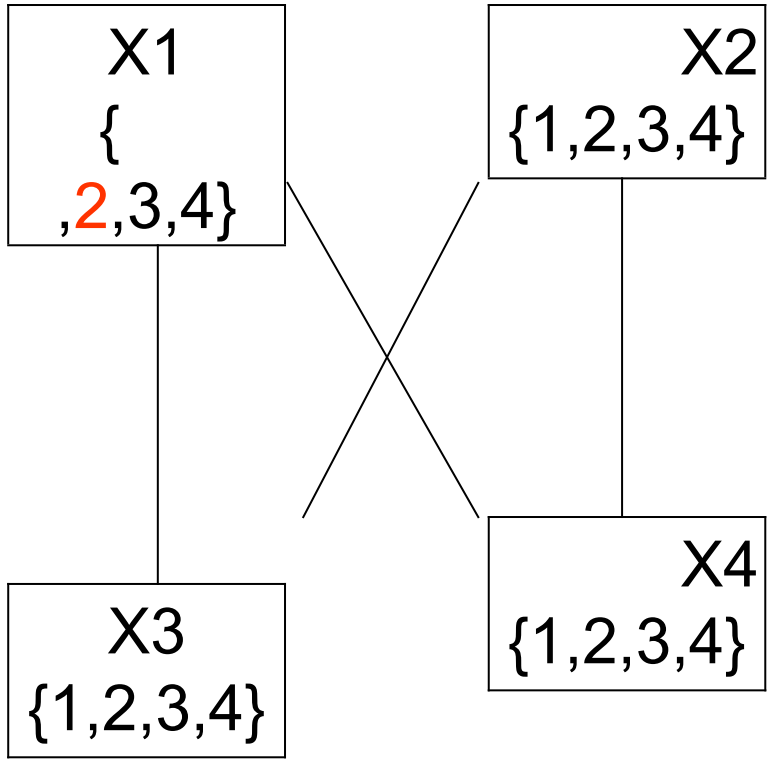
Example: 4-Queens Problem



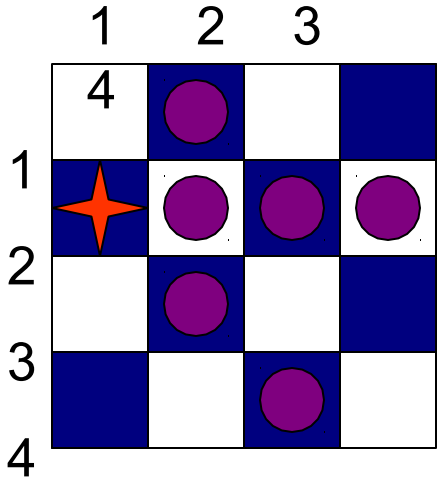


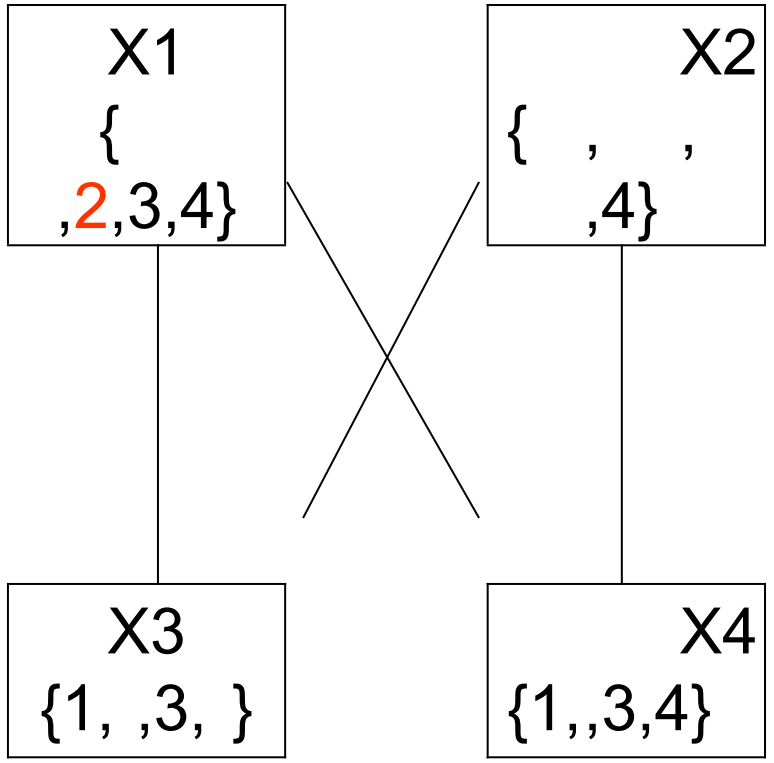
Example: 4-Queens Problem



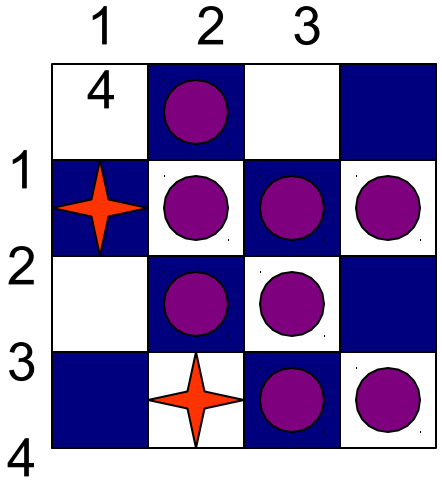


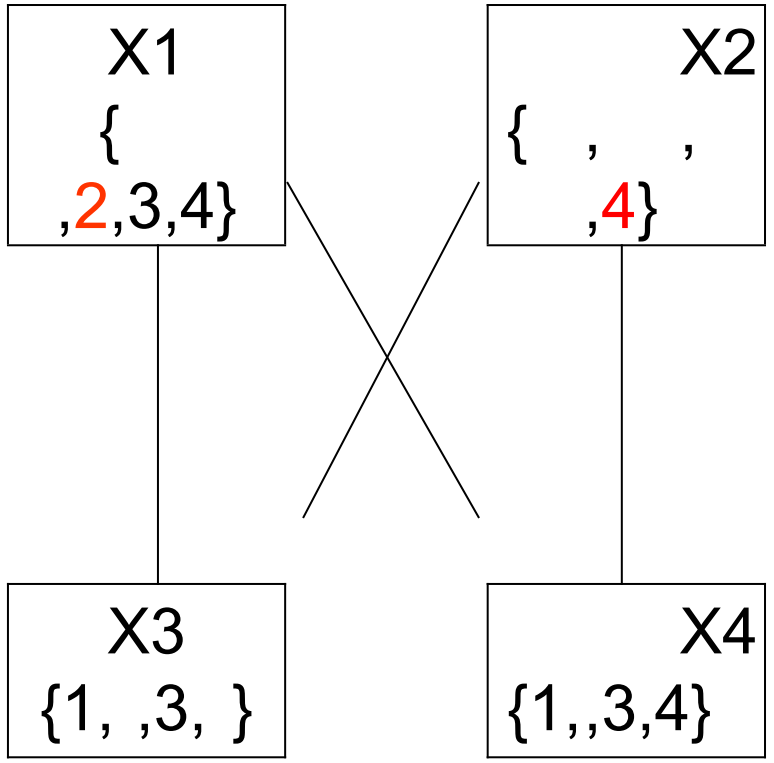
Example: 4-Queens Problem



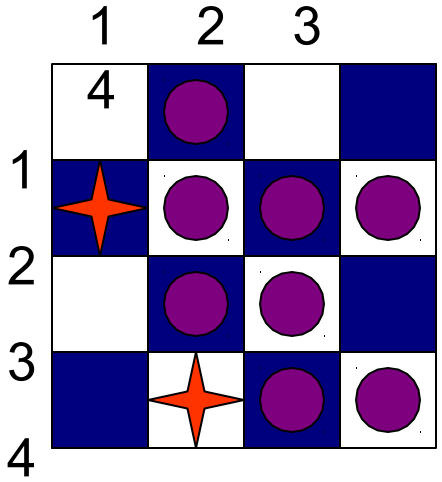


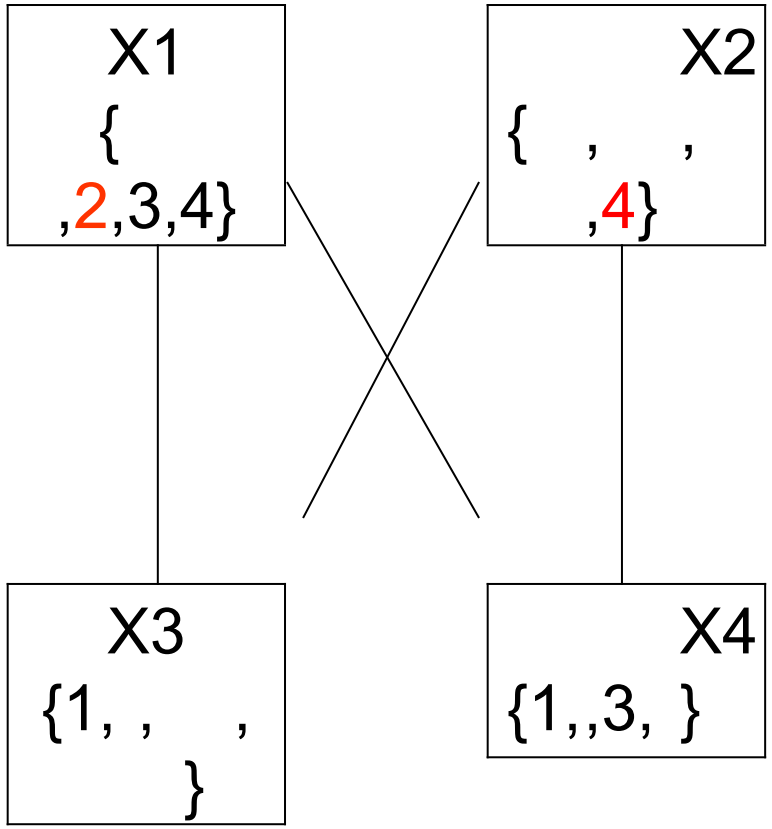
Example: 4-Queens Problem



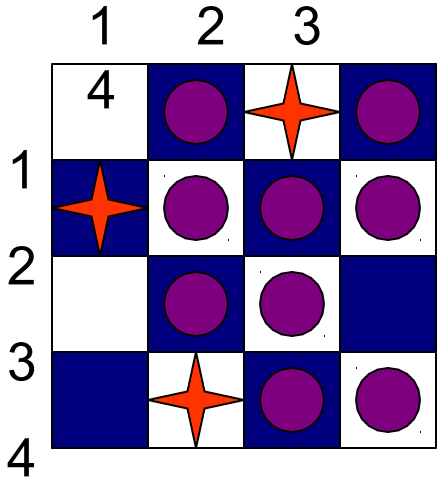


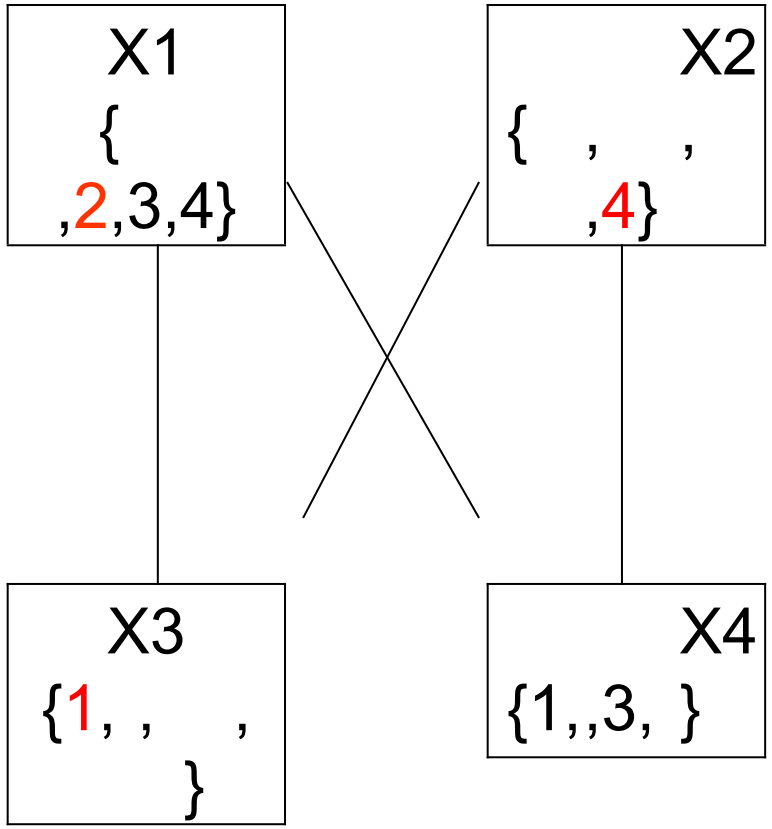
Example: 4-Queens Problem



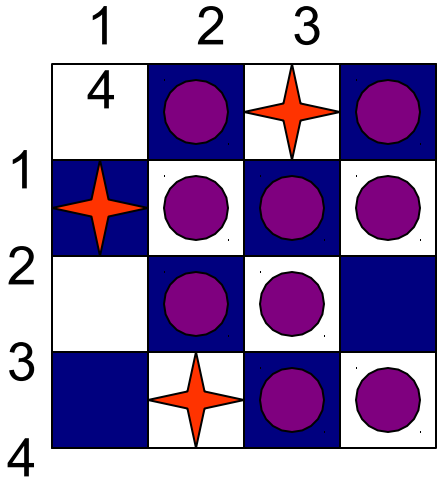


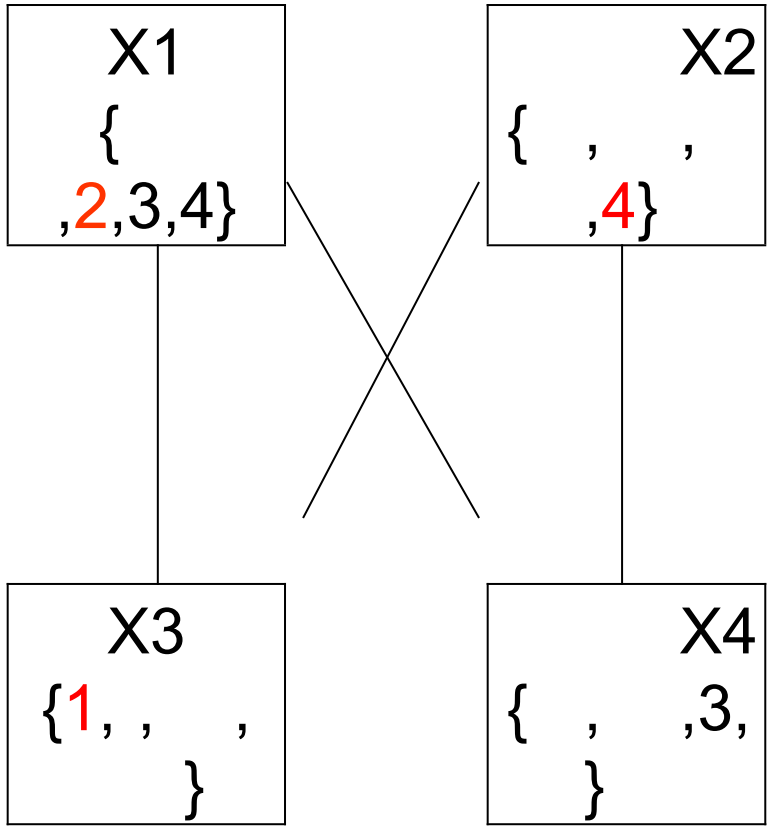
Example: 4-Queens Problem





Example: 4-Queens Problem





Example: 4-Queens Problem